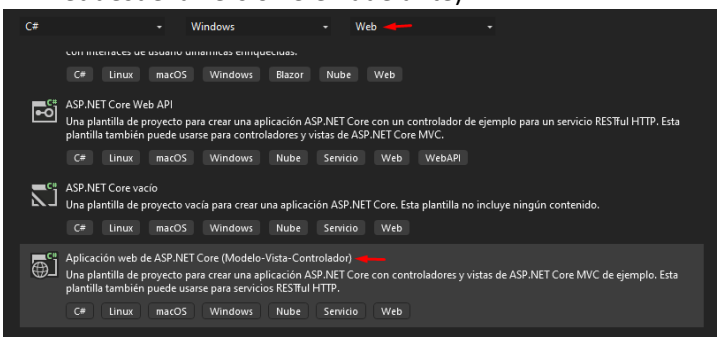
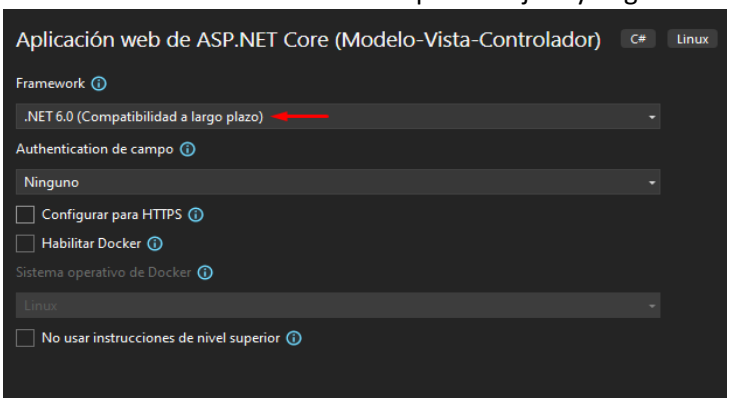
Aplicación Web MVC Net Core

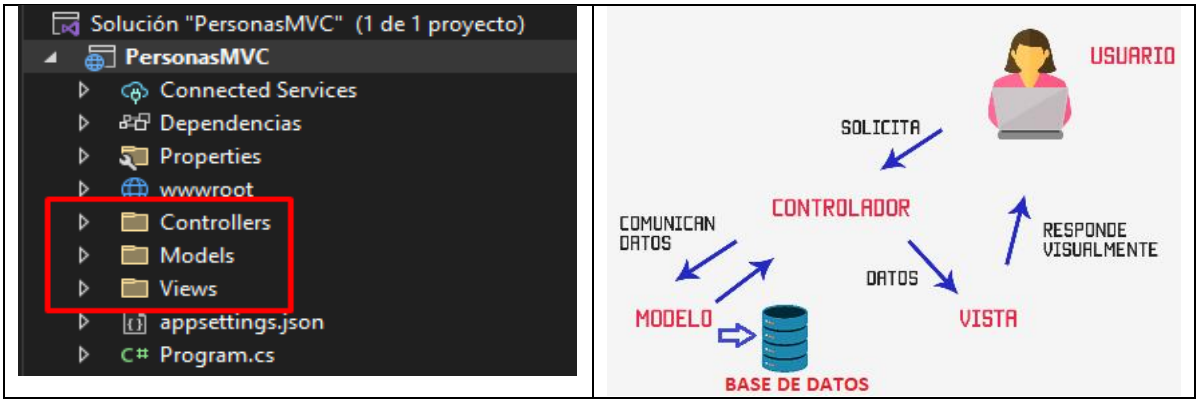
1. En Visual Studio Comunity crear una aplicación web MVC Net Core (Los lenguajes soportados son C#, F# y VB.Net desde la versión 3 en adelante)



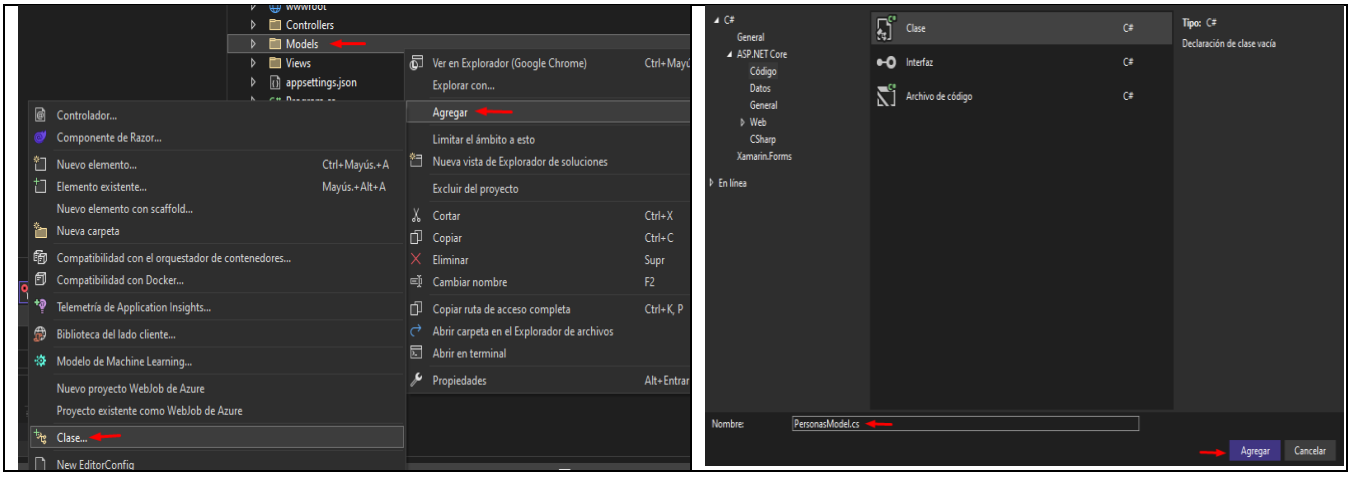
2. Selecciono la versión de Net Core que trabajare y hago clic en Crear



3. Al terminar de crear el proyecto MVC se crean las carpetas (Controllers-Models-Views)



4. Vamos a **Models** y creamos la clase **PersonasMode**l que representa la estructura de la tabla que está en la base de datos



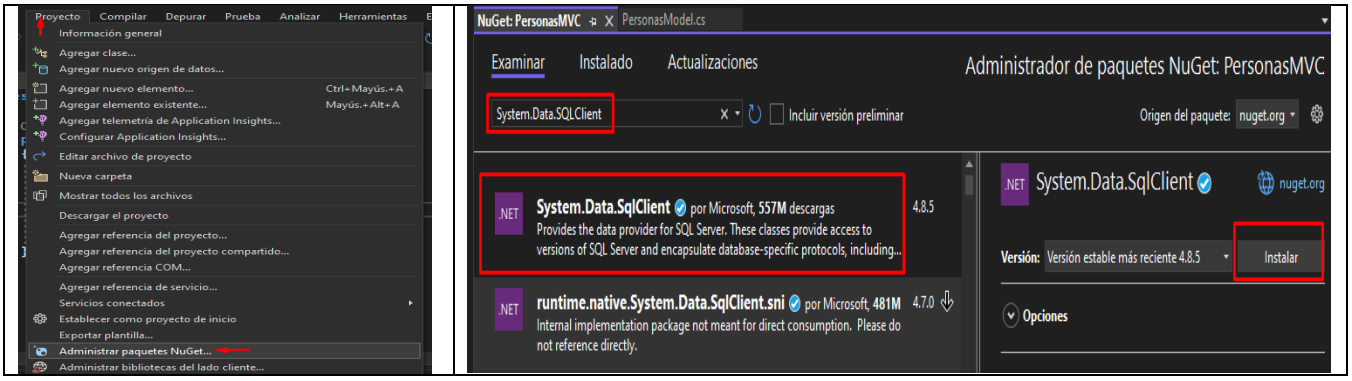
5. Dentro de la clase **PersonasModel** escribimos **prop + tab** para escribir las propiedades que tendrá nuestra clase, que deber ser la misma de la tabla

Estructura Tabla

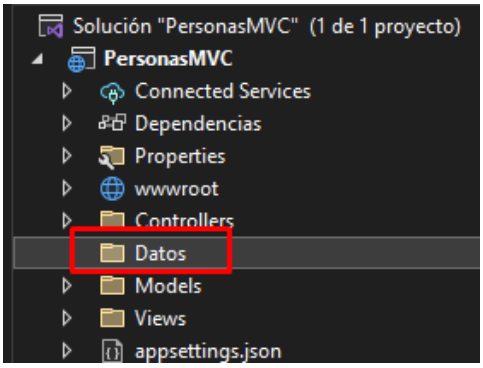
- dbo.Persona
  - Columnas
    - ID (PK, bigint, No NULL)
    - Nombre (varchar(250), NULL)
    - Telefono (varchar(50), NULL)

```
namespace PersonasMVC.Models
{
    0 referencias
    public class PersonasModel
    {
        0 referencias
        public long  ID { get; set; }
        0 referencias
        public string Nonbre { get; set; }
        0 referencias
        public string Telefono { get; set; }
    }
}
```

6. Ahora vamos agregar la librería **System.Data.SQLClient** para conectarme a la base de datos, para ellos vamos al Administrador de paquetes NuGet y lo instalamos



7. Creo en el proyecto una carpeta Datos para almacena la clase CRUDPersona que llamara los procedimientos almacenados



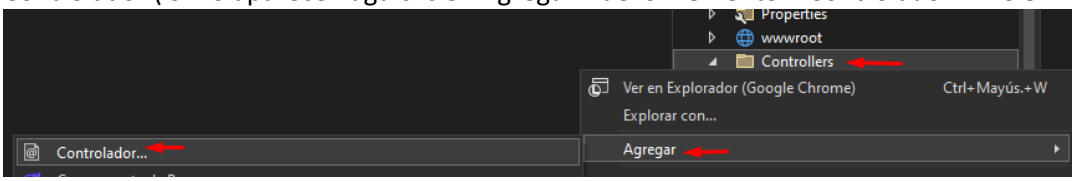
8. Creo la clase CRUDPersona en la carpeta Datos y escribo todo el código para conectar y ejecutar los procedimientos almacenados

```
public static bool Func_InsertarPersona(string nom, string tel)
{
    //convierto el json en un datatable
    DataTable dt = new DataTable();
    try
    {
        SqlConnection cnn = new SqlConnection(cadena);
        SqlDataAdapter adap = new SqlDataAdapter("PA_Insertar", cnn);
        adap.SelectCommand.CommandType = CommandType.StoredProcedure;
        adap.SelectCommand.Parameters.AddWithValue("@nom", nom);
        adap.SelectCommand.Parameters.AddWithValue("@tel", tel);
        adap.Fill(dt);
        return true;
    }
    catch
    {
        return false;
    }
}
```

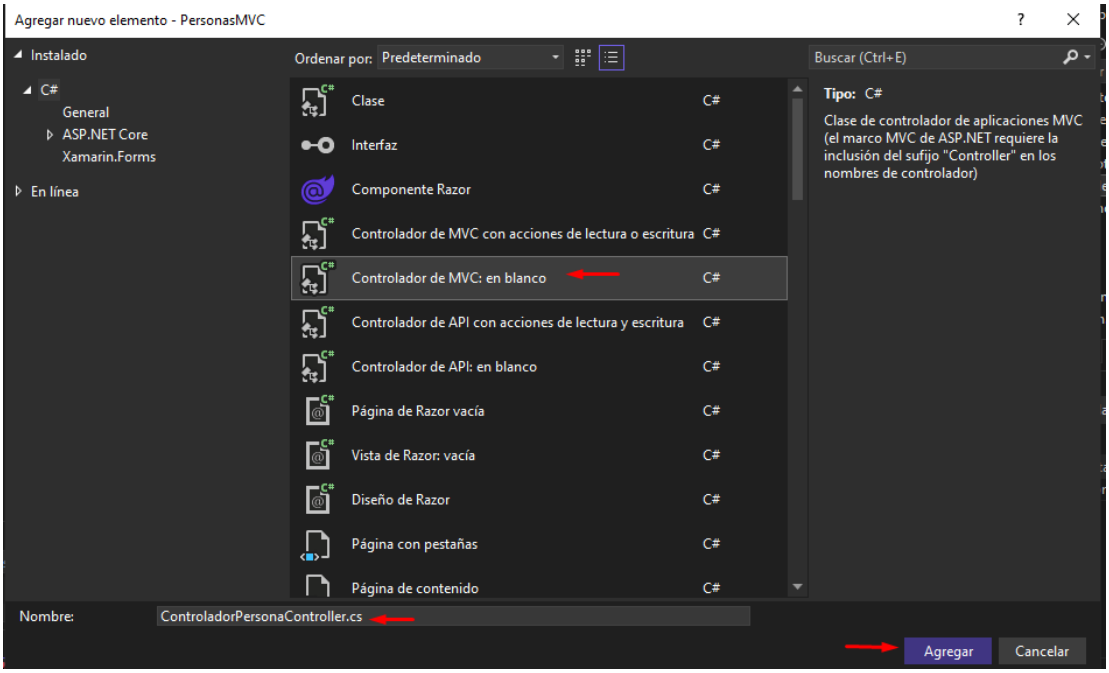
```
public static bool Func_EliminarPersona(long id)
{
    DataTable dt = new DataTable();
    try
    {
        SqlConnection cnn = new SqlConnection(cadena);
        SqlDataAdapter adap = new SqlDataAdapter("PA_Eliminar", cnn);
        adap.SelectCommand.CommandType = CommandType.StoredProcedure;
        adap.SelectCommand.Parameters.AddWithValue("@id", id);
        adap.Fill(dt);
        return true;

    }
    catch
    {
        return false;
    }
}
```

9. Damos clic en el carpeta **Controllers** con el botón izquierdo del mouse doy clic en Agregar y luego doy clic Controlador (Si no aparece haga clic en Agregar Nuevo Elemento – Controlador MVC en Blanco)



10. Selecciono Controlador MVC en Blanco, digito el nombre de controlador conservado siempre al final del nombre la palabra Controller con su extensión .cs y damos clic en Agregar



11. Haga clic en el controlador en **Index()** y con el lado derecho de mouse adicione una vista y después haga clic en vista Razor en Blanco, escriba el nombre **AgregarPesona**
12.
13. Haga la vista usando BootStrap 5 con las cajas de texto (nombre y teléfono), use el componente Card https://getbootstrap.com/docs/5.0/components/card/ y el componente Form https://getbootstrap.com/docs/5.0/forms/form-control/ el código debe quedar así:

```
@{
    <div class="card">
        <div class="card-header">
            Crear Persona
        </div>
        <div class="card-body">
            <form>
                <div class="mb-3">
                    <label for="exampleInputEmail1" class="form-label">Nombre</label>
                    <input type="email" class="form-control" id="exampleInputEmail1" aria-describedby="emailHelp">
                </div>
                <div class="mb-3">
                    <label for="exampleInputPassword1" class="form-label">Telefono</label>
                    <input type="password" class="form-control" id="exampleInputPassword1">
                </div>
                <button type="submit" class="btn btn-primary">Submit</button>
            </form>
        </div>
    </div>

}
```

14. Referenciaremos nuestro modelo usando @model **NombredelModelo** en la parte superior de la vista

```
@model PersonasModel
@{
    <div class="card">
        <div class="card-header">
            Crear Persona
        </div>
```

15. Referenciaremos la variables de modelo usando input asp-for="nombre variable modelo"

Antes

```
<div class="mb-3">
    <label for="exampleInputEmail1" class="form-label">Nombre</label>
    <input type="email" class="form-control" id="exampleInputEmail1" aria-describedby="emailHelp">
</div>
```

Después

```
<div class="mb-3">
    <label class="form-label">Nombre</label>
    <input asp-for="NombrePersona" class="form-control" >
</div>
```

16. Referenciaremos la acción o ruta que debe hacer el formulario usando form asp-action="accion o ruta del controlador" asp-controller="nombre controlador" method="metodo post"

```
@*accion-controlador-metodo*@
<form asp-action="GuardarPersona" asp-controller="Persona" method="post">
    <div class="mb-3">
        <label class="form-label">Nombre</label>
        <input asp-for="NombrePersona" class="form-control" >
    </div>
```

17. Hago clic en el Controlador, creo 2 acciones una para mostrar el formulario y otra para la acción que se ejecutara para mostrar el formulario y para Agregar una persona.

Muestra vista formulario

```
0 referencias
public IActionResult AgregarPersona()
{
    //Mostramos la vista del formulario
    return View();
}
```

Accion Metodo POST que se ejecutara

```
[HttpPost]
0 referencias
public IActionResult GuardarPersona(PersonaModel Modelo)
{
    //Accion Guardar Persona
    var model = new List<PersonaModel>();
    string nom = Modelo.NombrePersona;
    string tel = Modelo.TelefonoPersona;
    bool rpta = CrudClase.Func_Insertar(nom, tel);
    if (rpta == true)
    {
        return RedirectToAction("Index");

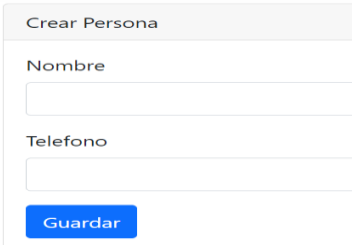
    }
    else
    {
        return View();
    }
}
```

18. Cambie la ruta de ejecucion de la aplicación web en Program.cs donde debe cambia en Action a nombre de la vista que iniciara.

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Persona}/{action=Agregar}/{id?}");

app.Run();
```

19. Ejecute la aplicación web y debe mostrar la vista para Agregar Personas, digite los datos y haga clic en Guardar, verifique si guarda en la base de datos.



20. Voy al controlador y agrego la acción que tendrá el index donde se creara una lista (List) la cual se llenara desde un Datatable

```
0 referencias
public IActionResult Index()
{
    var model = new List<PersonaModel>();
    DataTable dt = new DataTable();
    dt = CRUDPersonas.Func_TraerTodasPersonas();
    for (int i = 0; i < dt.Rows.Count; i++)
    {
        long id = Convert.ToInt64(dt.Rows[i]["ID"].ToString());
        string nombre = dt.Rows[i]["Nombre"].ToString();
        string telefono = dt.Rows[i]["Telefono"].ToString();
        //adiciono a la lista modelo
        model.Add(new PersonaModel { IDPersona = Convert.ToInt64(id), NombrePerona = nombre, TelefonoPersona = telefono });
    }
    //retorno la lista a la vista index
    return View(model);
}
```

21. Edito la vista Index , el modelo viene de una lista (List) del controlardor, se crea en enlace en la parte superior para crear una nueva persona y se crea una tabla con los datos que se mostraran

```
@model List<PersonaModel>
@{
    <a asp-controller="Persona" asp-action="Agregar" class="btn btn-primary">Crear</a>
    <table class="table">
        <thead>
            <tr>
                <th scope="col">ID</th>
                <th scope="col">Nombre</th>
                <th scope="col">Telefono</th>
            </tr>
        </thead>
```

22. En el Tbody de la tabla recorremos los datos de la lista (List) con un ForEach y mostramos ese contenido en la tabla referenciándolo con el modelo (@item) adicionamos las acciones de los botones Editar y Eliminar, recuerde que para cada botón se enviara el ID (asp-route-ID)

```
<tbody>
    @foreach (var item in Model)
    {
        <tr>
            <td>@item.IDPersona</td>
            <td>@item.NombrePerona</td>
            <td>@item.TelefonoPersona</td>
            <td>
                <a asp-action="Editar" asp-controller="Persona" class="btn btn-primary btn-sm" asp-route-ID="@item.IDPersona">Editar</a>
                <a asp-action="EliminarPersona" asp-controller="Persona" class="btn btn-danger btn-sm"  asp-route-ID="@item.IDPersona">Eliminar</a>
            </td>
        </tr>
    }
</tbody>
</table>
```

23. Cambie la ruta de ejecucion de la aplicación web en Program.cs donde debe cambia en Action a nombre de la vista que iniciara en este caso Index

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Persona}/{action=Index}/{id?}");
```

24. Ejecute la aplicación web y debe mostrar la vista Index con el listado de las personas que ya se hayan agregado

Crear

ID	Nombre	Telefono	
9	Bladimir Silva	12345	Editar Eliminar
14	Juan Juan	9999999999	Editar Eliminar
15	juan camilo	123434	Editar Eliminar

25. Vaya al controlador y realice la acción para eliminar una persona

```
public IActionResult EliminarPersona(long id)
{
    if (CRUDPersonas. Func_EliminarPersona(id))
    {
        return RedirectToAction("Index");
    }
    else
    {
        return View();
    }
}
```

26. Hago clic en el Controlador, creo 2 acciones una para mostrar el formulario y otra para la acción que se ejecutara para mostrar el formulario y para Editar una persona.

Muestra vista formulario

Consulta un persona con su ID esa informacion es traida en un DataTable y se llena el modelo con dicha informacion

```
public IActionResult Editar(long id)
{
    var model = new PersonaModel();
    DataTable dt = new DataTable();
    dt = CRUDPersonas.Func_TraerUnaPersona(id);
    for (int i = 0; i < dt.Rows.Count; i++)
    {
        model.IDPersona = id;
        model.NombrePerona = dt.Rows[i]["Nombre"].ToString();
        model.TelefonoPersona = dt.Rows[i]["Telefono"].ToString();
    }
    return View(model);
}
```

Accion Metodo POST que se ejecutara

```
[HttpPost]
0 referencias
public IActionResult EditarPersona(PersonaModel Modelo)
{
    var lista = new List<PersonaModel>();
    long id = Modelo.IDPersona;
    string nom = Modelo.NombrePerona;
    string tel = Modelo.TelefonoPersona;
    bool rpta = CRUDPersonas.Func_EditarPersona(nom, tel,id);
    if (rpta == true)
    {
        return RedirectToAction("Index");
    }
    else
    {
        return View();
    }

}
```

27. Voy a las vistas y agrego la vista que tendrá la acción Editar

```
@model PersonaModel
@{
    <div class="card">
        <div class="card-header">
            Editar Persona
        </div>
        <div class="card-body">
            <form asp-action="EditarPersona" asp-controller="Persona" method="post">
                <div class="mb-3">
                    <label class="form-label">ID</label>
                    <input asp-for="IDPersona" class="form-control" value=@Model.IDPersona readonly>
                </div>
                <div class="mb-3">
                    <label class="form-label">Nombre</label>
                    <input asp-for="NombrePerona" class="form-control" value=@Model.NombrePerona>
                </div>
                <div class="mb-3">
                    <label class="form-label">Telefono</label>
                    <input asp-for="TelefonoPersona" class="form-control" value=@Model.TelefonoPersona>
                </div>
                <button type="submit" class="btn btn-primary">Editar</button>
            </form>
        </div>
    </div>
```

Uso de sweetalert2

28. Para esto vamos al Index y al final escribiremos este código mezclando JavaScript con C#

```
<script src="https://cdn.jsdelivr.net/npm/sweetalert2@11"></script>
@if (TempData["mensaje"] != null)
{
    <script>
        Swal.fire({
            title: '@TempData["titulo"].ToString()',
            text: '@TempData["mensaje"].ToString()',
            icon: '@TempData["icono"].ToString()',
            confirmButtonText: 'OK'
        })
    </script>
}
```

29. Para llamarlo desde el controlador escribiremos este código en C#

```
//mensaje sweetalert
TempData["mensaje"] = "Cliente Creado Correctamente!!!.";
TempData["titulo"] = "Correcto.";
TempData["icono"] = "success";
```